

</>

The Anuj DECODED

</>

```

01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};

```



```

01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver

```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

3483 · Unique 3-Digit Even Numbers

LeetCode · Daily Question

Easy

Counting

Frequency Array

1. What Type of Question Is This?

This is a **constrained counting / combinatorics** problem dressed up as a permutation problem. On the surface it sounds like "count arrangements", but what actually matters is only **how many copies of each digit value (0–9) are available** — not which physical array positions they came from. Whenever duplicates in the input only matter by *count*, not by identity, the standard move is to collapse the input into a small **frequency array** (here, size 10) and reason directly in terms of digit values instead of array indices.

2. What Is Given?

- An array **digits** of single digits (0–9), with 3 to 10 entries.
- You must form 3-digit **even** numbers using these digits, with **no leading zero**.
- Each *copy* of a digit can be used **once per number** — so if a digit appears twice in the array, one formed number may use that digit twice (as long as it's in two different positions), but not three times unless it appears three times in the array.

3. What Do We Need to Find?

The count of **distinct** 3-digit even numbers that can be formed — if the same numeric value can be built in several different ways (different positions or a different physical copy of a repeated digit), it's still only counted **once**.

Function Signature

■ C++

```
class Solution {
public:
    int totalNumbers(vector<int>& digits) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  digits = [1,2,3,4]
Output: 12
```

Every digit here appears exactly once, so no digit can be reused within a single number. The 12 distinct even numbers are 124, 132, 134, 142, 214, 234, 312, 314, 324, 342, 412, and 432. Note that 222 is impossible — there's only one copy of the digit 2 available.

Example 2 — a repeated digit

```
Input:  digits = [0,2,2]
Output: 2
```

Only 202 and 220 work. The digit 2 can be used **twice** in the same number because the array contains two copies of it — but 0 can only be used once (only one copy available), and it can never lead.

Example 3 — all three copies of one digit

```
Input:  digits = [6,6,6]
Output: 1
```

Only 666 can be formed — there are exactly three copies of 6 available, exactly enough for one 3-digit number using it in every position.

Example 4 — no even digit available

```
Input:  digits = [1,3,5]
Output: 0
```

Every available digit is odd, so no number ending in an even digit can ever be formed → **0**.

Constraints

- $3 \leq \text{digits.length} \leq 10$
- $0 \leq \text{digits}[i] \leq 9$

`digits.length` is tiny (at most 10), so almost any approach will run fast enough in practice — but the frequency-array technique below is worth internalizing since it scales to far larger inputs and completely sidesteps the need to de-duplicate results.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Realize the array's exact layout doesn't matter.** Whether the input is [2,2] or [2,2] arranged differently makes no difference — all that matters is "how many 2's are available". Collapse digits into a length-10 **frequency array** `count[0..9]`.
- **Step 2 — Notice a 3-digit number is fully determined by 3 digit values, not by which array element supplied them.** Once you fix (hundreds digit, tens digit, units digit) as specific values 0–9, you've fixed one specific number — there's no way to double count it by iterating over digit *values* instead of array *positions*. This means a set for deduplication becomes unnecessary if you search over values directly.
- **Step 3 — Add the constraints directly into the search.** The units digit must be even (0, 2, 4, 6, or 8) and have at least one copy available. The hundreds digit must be non-zero and have at least one copy available *after* accounting for whatever the units digit already used. The tens digit can be anything with at least one copy available after both other choices are accounted for.
- **Step 4 — Use temporary decrement/restore (a tiny backtrack) to track availability.** When tentatively choosing a digit for one position, decrement its count so the next position's check correctly reflects how many copies are left; restore it afterward before trying the next candidate.

Which Algorithm Is Best?

Searching directly over the 10 possible digit values for each of the 3 positions (with counts decremented/restored along the way) checks at most $10 \times 10 \times 5 = 500$ combinations, **completely independent of digits.length**. This is the cleanest, fastest, and most memory-light solution: no permutation generation, and no set needed for deduplication since each (hundreds, tens, units) triple already corresponds to exactly one distinct number. The brute-force and "better" tiers below show why searching over array *positions* instead of digit *values* costs much more for no benefit.

Approach 1 — Brute Force (Permute the Whole Array, Look at the First 3 Slots)

Idea: The most literal reading of "arrange these digits" is to generate every full permutation of the entire input array, and for each one just look at the first three positions to form a candidate number. A set removes duplicate results.

Steps

- Generate every permutation of the full digits array.
- For each permutation, take its first 3 elements as (hundreds, tens, units).
- Reject it if the hundreds digit is 0, or the units digit is odd.
- Otherwise compute the 3-digit number and insert it into a set.
- Return the set's final size.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int totalNumbers(vector<int>& digits) {
        sort(digits.begin(), digits.end());
        set<int> seen;

        // Permute the ENTIRE array, then just look at its first 3 positions.
        do {
            if (digits[0] == 0) continue;           // leading zero
            if (digits[2] % 2 != 0) continue;       // must be even
            int num = digits[0] * 100 + digits[1] * 10 + digits[2];
            seen.insert(num);
        } while (next_permutation(digits.begin(), digits.end()));

        return (int)seen.size();
    }
};

int main() {
    vector<int> digits = {1, 2, 3, 4};
    Solution sol;
    cout << sol.totalNumbers(digits) << endl;    // Expected output: 12
    return 0;
}
```

■ Python

```
from itertools import permutations

class Solution:
    def totalNumbers(self, digits):
        seen = set()

        # Permute the ENTIRE array, then just look at its first 3 positions.
        for perm in set(permutations(digits)):
            if perm[0] == 0:
                continue           # leading zero
            if perm[2] % 2 != 0:
                continue           # must be even
            num = perm[0] * 100 + perm[1] * 10 + perm[2]
```

```

        seen.add(num)

        return len(seen)

if __name__ == "__main__":
    digits = [1, 2, 3, 4]
    print(Solution().totalNumbers(digits))    # Expected output: 12

```

■ JavaScript

```

class Solution {
    totalNumbers(digits) {
        const seen = new Set();
        const n = digits.length;
        const used = new Array(n).fill(false);
        const perm = [];

        // Permute the ENTIRE array, then just look at its first 3 positions.
        const permute = () => {
            if (perm.length === n) {
                if (perm[0] !== 0 && perm[2] % 2 === 0) {
                    seen.add(perm[0] * 100 + perm[1] * 10 + perm[2]);
                }
                return;
            }
            for (let i = 0; i < n; i++) {
                if (used[i]) continue;
                used[i] = true;
                perm.push(digits[i]);
                permute();
                perm.pop();
                used[i] = false;
            }
        };

        permute();
        return seen.size;
    }
}

const digits = [1, 2, 3, 4];
console.log(new Solution().totalNumbers(digits)); // Expected output: 12

```

Complexity: Permuting all n elements costs $O(n!)$ permutations, each needing $O(n)$ to generate — $O(n \cdot n!)$ overall. With `digits.length` up to 10, that's up to ~36 million operations from permutations whose 4th-through-10th positions are completely irrelevant to the answer — correct, but enormously wasteful. (This implementation also de-duplicates identical permutations first, since standard permutation generators don't know about repeated values.)

Approach 2 — Better (Only Pick 3 Positions, Stop Permuting the Rest)

Idea: Recognize that positions 4 through n never affect the answer at all — stop generating them. Instead directly search for an ordered choice of exactly 3 distinct array positions (a partial permutation), which is naturally far smaller than permuting everything.

Steps

- Backtrack over array positions, picking 3 distinct (not-yet-used) indices in order.
- The instant 3 positions are chosen, check the leading-zero and even-units conditions and record the resulting number in a set if valid.
- Undo the last choice (mark it unused again) and try the next candidate position.
- Return the set's size once the search finishes.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int totalNumbers(vector<int>& digits) {
        int n = digits.size();
        set<int> seen;
        vector<bool> used(n, false);

        // Only pick exactly 3 positions (a "partial permutation"), instead of permuting
        // every position in the array.
        function<void(vector<int>&)> pick = [&](vector<int>& chosen) {
            if (chosen.size() == 3) {
                if (chosen[0] == 0) return;           // leading zero
                if (chosen[2] % 2 != 0) return;       // must be even
                seen.insert(chosen[0] * 100 + chosen[1] * 10 + chosen[2]);
                return;
            }
            for (int i = 0; i < n; i++) {
                if (used[i]) continue;
                used[i] = true;
                chosen.push_back(digits[i]);
                pick(chosen);
                chosen.pop_back();
                used[i] = false;
            }
        };

        vector<int> chosen;
        pick(chosen);
        return (int)seen.size();
    }
};

int main() {
    vector<int> digits = {1, 2, 3, 4};
    Solution sol;
    cout << sol.totalNumbers(digits) << endl;    // Expected output: 12
    return 0;
}
```

■ Python

```
class Solution:
```

```

def totalNumbers(self, digits):
    n = len(digits)
    seen = set()
    used = [False] * n

    # Only pick exactly 3 positions (a "partial permutation"), instead of permuting
    # every position in the array.
    def pick(chosen):
        if len(chosen) == 3:
            if chosen[0] == 0:
                return # leading zero
            if chosen[2] % 2 != 0:
                return # must be even
            seen.add(chosen[0] * 100 + chosen[1] * 10 + chosen[2])
            return
        for i in range(n):
            if used[i]:
                continue
            used[i] = True
            chosen.append(digits[i])
            pick(chosen)
            chosen.pop()
            used[i] = False

    pick([])
    return len(seen)

if __name__ == "__main__":
    digits = [1, 2, 3, 4]
    print(Solution().totalNumbers(digits)) # Expected output: 12

```

■ JavaScript

```

class Solution {
    totalNumbers(digits) {
        const n = digits.length;
        const seen = new Set();
        const used = new Array(n).fill(false);
        const chosen = [];

        // Only pick exactly 3 positions (a "partial permutation"), instead of permuting
        // every position in the array.
        const pick = () => {
            if (chosen.length === 3) {
                if (chosen[0] !== 0 && chosen[2] % 2 === 0) {
                    seen.add(chosen[0] * 100 + chosen[1] * 10 + chosen[2]);
                }
                return;
            }
            for (let i = 0; i < n; i++) {
                if (used[i]) continue;
                used[i] = true;
                chosen.push(digits[i]);
                pick();
                chosen.pop();
                used[i] = false;
            }
        };

        pick();
        return seen.size;
    }
}

const digits = [1, 2, 3, 4];
console.log(new Solution().totalNumbers(digits)); // Expected output: 12

```

Complexity: Choosing an ordered sequence of 3 distinct positions out of n costs $O(n \times (n-1) \times (n-2))$ — at most $10 \times 9 \times 8 = 720$ combinations, dramatically fewer than Approach 1's full permutations. Still needs a set to remove duplicate numeric results, since two different position-triples can spell the same number when a digit value repeats

in the array.

Approach 3 — Optimal (Search Over Digit Values 0–9, Not Array Positions)

Idea: Collapse the input into a length-10 frequency array once, then directly search over which *digit values* (not array positions) occupy the hundreds, tens, and units places, decrementing and restoring counts as a tiny 3-level backtrack. Since the search is over digit values, each (hundreds, tens, units) triple already corresponds to exactly one distinct number — no set needed anywhere.

Steps

- Build `count[0..9]`: how many copies of each digit value are available.
- For each even units digit (0, 2, 4, 6, 8) with at least one copy available: tentatively use one copy (decrement `count[units]`).
- For each non-zero hundreds digit with at least one copy available *after* that: tentatively use one copy (decrement `count[hundreds]`).
- For each tens digit 0–9 with at least one copy still available: this is one more valid, automatically distinct 3-digit number — increment the total.
- Restore `count[hundreds]`, then restore `count[units]`, and continue to the next candidate.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int totalNumbers(vector<int>& digits) {
        int count[10] = {0};
        for (int d : digits) count[d]++;

        int total = 0;

        for (int units = 0; units <= 8; units += 2) { // must be even
            if (count[units] == 0) continue;
            count[units]--;

            for (int hundreds = 1; hundreds <= 9; hundreds++) { // no leading zero
                if (count[hundreds] == 0) continue;
                count[hundreds]--;

                for (int tens = 0; tens <= 9; tens++) {
                    if (count[tens] > 0) total++; // one valid distinct number
                }

                count[hundreds]++; // restore
            }
            count[units]++; // restore
        }

        return total;
    }
};

int main() {
    vector<int> digits = {1, 2, 3, 4};
    Solution sol;
    cout << sol.totalNumbers(digits) << endl; // Expected output: 12
    return 0;
}
```

■ Python

```
class Solution:
    def totalNumbers(self, digits):
        count = [0] * 10
        for d in digits:
            count[d] += 1

        total = 0

        for units in range(0, 9, 2):          # must be even: 0, 2, 4, 6, 8
            if count[units] == 0:
                continue
            count[units] -= 1

            for hundreds in range(1, 10):    # no leading zero
                if count[hundreds] == 0:
                    continue
                count[hundreds] -= 1

                for tens in range(10):
                    if count[tens] > 0:
                        total += 1           # one valid distinct number

                count[hundreds] += 1         # restore

            count[units] += 1               # restore

        return total

if __name__ == "__main__":
    digits = [1, 2, 3, 4]
    print(Solution().totalNumbers(digits)) # Expected output: 12
```

■ JavaScript

```
class Solution {
    totalNumbers(digits) {
        const count = new Array(10).fill(0);
        for (const d of digits) count[d]++;

        let total = 0;

        for (let units = 0; units <= 8; units += 2) {          // must be even
            if (count[units] === 0) continue;
            count[units]--;

            for (let hundreds = 1; hundreds <= 9; hundreds++) { // no leading zero
                if (count[hundreds] === 0) continue;
                count[hundreds]--;

                for (let tens = 0; tens <= 9; tens++) {
                    if (count[tens] > 0) total++;                // one valid distinct number
                }

                count[hundreds]++;                                // restore
            }
            count[units]++;                                        // restore
        }

        return total;
    }
}

const digits = [1, 2, 3, 4];
console.log(new Solution().totalNumbers(digits)); // Expected output: 12
```

Complexity: The search only ever considers 10 possible digit values for each position: at most $5 \times 9 \times 10 = 450$ (units $\times$ hundreds $\times$ tens) combinations, completely **independent of digits.length**. This is effectively $O(1)$ time and $O(1)$ extra space (a fixed 10-entry array), and needs no deduplication step at all.

Side-by-Side Comparison

1. Brute Force	Permute the whole array, look only at the first 3 slots, deduplicate	$O(n \cdot n!)$	$O(n!)$ for dedup set
2. Better	Choose an ordered 3-position partial permutation directly, deduplicate	$O(n \cdot (n-1) \cdot (n-2))$	(result count) for dedup set
3. Optimal	Search over the 10 digit VALUES directly using a frequency array	$O(1)$ — independent of n	$O(1)$ — fixed size-10

Key Takeaways to Remember

- **When duplicates only matter by count, not identity**, collapse the input into a small frequency array indexed by value, and search over *values* instead of *positions*.
- Searching over a fixed, small value range (here, digits 0–9) instead of over the input's positions can make the algorithm's complexity **independent of input size entirely** — a genuine change in complexity class, not just a smaller constant.
- A results set for deduplication is only needed when your search can reach the *same outcome* via multiple different paths. Searching over digit values directly removes that possibility altogether, since each combination of values already *is* the distinct result.
- A tentative decrement-then-restore around a recursive/nested step is a lightweight, general backtracking pattern for "use one unit of a limited resource, then give it back" — useful far beyond this specific problem.
- All three tiers were validated against **Example 1** (digits = [1,2,3,4] → 12) across C++, Python, and JavaScript, and checked against Examples 2, 3, and 4 as well.